B.Tech. BCSE497J - Project-I

AN EXPLAINABLE MULTI-MODEL MACHINE LEARNING FRAMEWORK FOR FINANCIAL FRAUD DETECTION WITH SECURE API INTEGRATION

Submitted in partial fulfillment of the requirements for the degree of

Bachelor of Technology

in

Computer Science and Engineering

by

23BCE0374	**DHRUV SINGH**
23BCE0415	**ANIMESH B. LABH**
23BDS0091	**KUSHAGRA DHELIA**

Under the Supervision of

Dr. Erapaneni Gayatri

Assistant Professor Sr. Grade 1

School of Computer Science and Engineering (SCOPE)



Vellore Institute of Technology

(Deemed to be University under section 3 of UGC Act, 1956)

October 2026

DECLARATION

I hereby declare that the project entitled **AN EXPLAINABLE MULTI-MODEL MACHINE LEARNING FRAMEWORK FOR FINANCIAL FRAUD DETECTION WITH SECURE API INTEGRATION** submitted by us, for the award of the degree of *Bachelor of Technology in Computer Science and Engineering* to VIT is a record of bonafide work carried out by us under the supervision of **Dr. Erapaneni Gayatri**, Assistant Professor Sr. Grade 1, School of Computer Science and Engineering (SCOPE), Vellore Institute of Technology, Vellore.

I further declare that the work reported in this project has not been submitted and will not be submitted, either in part or in full, for the award of any other degree or diploma in this institute or any other institute or university.

Place : Vellore

Date : 26 October 2026

Signature of the Candidates:

1. **Dhruv Singh (23BCE0374)**
2. **Animesh B. Labh (23BCE0415)**
3. **Kushagra Dhelia (23BDS0091)**

CERTIFICATE

This is to certify that the project entitled **AN EXPLAINABLE MULTI-MODEL MACHINE LEARNING FRAMEWORK FOR FINANCIAL FRAUD DETECTION WITH SECURE API INTEGRATION** submitted by **DHRUV SINGH (23BCE0374), ANIMESH B. LABH (23BCE0415), and KUSHAGRA DHELIA (23BDS0091)**, School of Computer Science and Engineering, VIT, for the award of the degree of *Bachelor of Technology in Computer Science and Engineering*, is a record of bonafide work carried out by them under my supervision during Fall Semester 2026-2027, as per the VIT code of academic and research ethics.

The contents of this report have not been submitted and will not be submitted either in part or in full, for the award of any other degree or diploma in this institute or any other institute or university. The project fulfills the requirements and regulations of the University and in my opinion meets the necessary standards for submission.

Place : Vellore

Date : 26 October 2026

Signature of the Guide

Dr. Erapaneni Gayatri
Assistant Professor Sr. Grade 1
School of Computer Science and Engineering (SCOPE)

Examiner(s):
1.
2.

Head of the Department
Dr. Boominathan P
Department of Computer Science and Engineering
School of Computer Science and Engineering (SCOPE)

ACKNOWLEDGEMENTS

I am deeply grateful to the management of **Vellore Institute of Technology (VIT)** for providing me with the opportunity and resources to undertake this project. Their commitment to fostering a conducive learning environment has been instrumental in my academic journey. The support and infrastructure provided by VIT have enabled me to explore and develop my ideas to their fullest potential.

My sincere thanks to **Dr. Jaisankar N**, the Dean of the School of Computer Science and Engineering (SCOPE), for his unwavering support and encouragement. His leadership and vision have greatly inspired me to strive for excellence. The Dean's dedication to academic excellence and innovation has been a constant source of motivation for me. I appreciate his efforts in creating an environment that nurtures creativity and critical thinking.

I express my profound appreciation to **Dr. Boominathan P**, the Head of the Department of Computer Science and Engineering, for his insightful guidance and continuous support. His expertise and advice have been crucial in shaping the direction of my project. The Head of Department's commitment to fostering a collaborative and supportive atmosphere has greatly enhanced my learning experience. His constructive feedback and encouragement have been invaluable in overcoming challenges and achieving my project goals.

I am immensely thankful to my project guide, **Dr. Erapaneni Gayatri**, Assistant Professor Sr. Grade 1, for her dedicated mentorship and invaluable feedback. Her patience, knowledge, and encouragement have been pivotal in the successful completion of this project. My supervisor's willingness to share her expertise and provide thoughtful guidance has been instrumental in refining my ideas and methodologies. Her support has not only contributed to the success of this project but has also enriched my overall academic experience.

Thank you all for your contributions and support.

Dhruv Singh (23BCE0374)

Animesh B. Labh (23BCE0415)

Kushagra Dhelia (23BDS0091)

TABLE OF CONTENTS

Sl.No	Contents	Page No.
	Abstract	**x**
1.	**INTRODUCTION**	**1**
	1.1 Background	1
	1.2 Motivations	1
	1.3 Scope of the Project	2
2.	**PROJECT DESCRIPTION AND GOALS**	**3**
	2.1 Literature Review	3
	2.2 Research Gap	4
	2.3 Objectives	4
	2.4 Problem Statement	5
	2.5 Project Plan	5
3.	**TECHNICAL SPECIFICATION**	**6**
	3.1 Requirements	6
	3.1.1 Functional Requirements	6
	3.1.2 Non-Functional Requirements	6
	3.2 Feasibility Study	6
	3.2.1 Technical Feasibility	6
	3.2.2 Economic Feasibility	7
	3.2.3 Social Feasibility	7
	3.3 System Specification	7
	3.3.1 Hardware Specification	7
	3.3.2 Software Specification	7
4.	**DESIGN APPROACH AND DETAILS**	**9**
	4.1 System Architecture	9
	4.2 Design	10
	4.2.1 Data Flow Diagram	10
	4.2.2 Use Case Diagram	10
	4.2.3 Class Diagram	11
	4.2.4 Sequence Diagram	12

Sl.No	Contents (Continued)	Page No.
5.	**METHODOLOGY AND TESTING**	**13**
	5.1 Module Description	13
	5.1.1 Preprocessing and Feature Forensics	13
	5.1.2 Class Imbalance Handling via SMOTE-NC	13
	5.1.3 Supervised Tree Ensembles (RF & XGBoost)	14
	5.1.4 Deep Neural Network & Autoencoder Representation	14
	5.1.5 Multi-Model Consensus Voting & Scoring Engine	14
	5.1.6 Explainability via TreeSHAP and Balance Forensics	15
	5.1.7 Hardened RESTful API & Security Subsystem	15
	5.1.8 Real-Time Kafka Event Streaming Pipeline	15
	5.2 Testing	16
	5.2.1 Automated Test Suite & Regression Testing	16
	5.2.2 Model Integrity & Preprocessing Edge Cases	17
	5.2.3 Security, Authorization & Injection Resistance	17
	5.2.4 Latency & Concurrency Stress Testing	18
6.	**PROJECT DEMONSTRATION**	**19**
	6.1 Multi-Tenant Streamlit Analyst Workspace	19
	6.2 Single-Transaction Scoring & Forensic Diagnosis	20
	6.3 Batch CSV Inference & Audit Trail	20
	6.4 Explainable AI Visualizations	20
	6.5 Live & Simulated Kafka Streaming Feeds	21
7.	**RESULT AND DISCUSSION (COST ANALYSIS)**	**22**
	7.1 Primary Benchmark Evaluation (3,000 Holdout)	22
	7.2 Secondary Benchmark & Cross-Generator Shift Analysis	23
	7.3 Precision-Recall & ROC Analysis	24
	7.4 False Positive Reduction & Operational Tradeoffs	25
	7.5 Financial Cost-Benefit & Economic Loss Modeling	26
8.	**CONCLUSION**	**27**
	8.1 Summary of Contributions	27
	8.2 Limitations	27
	8.3 Future Work	27
9.	**REFERENCES**	**28**
	APPENDIX A - SAMPLE CODE	**29**

List of Figures

Figure No.	Title	Page No.
4.1	Architecture of the Multi-Model Consensus Fraud Scoring Pipeline	9
4.2	Data Flow Diagram (DFD Level 1) of the Transaction Processing Engine	10
4.3	Use Case Diagram for Enterprise Fraud Detection and Analyst Workspace	10
4.4	Class Diagram of Preprocessing, Consensus, and Streaming Components	11
4.5	Sequence Diagram of Transaction Ingestion, Scoring, and Audit Logging	12
4.6	Event-Driven Apache Kafka Real-Time Streaming Ingestion Pipeline	16
6.1	Class Distribution Before and After Synthetic Minority Oversampling (SMOTE)	19
6.2	Training and Validation Loss Curves for Deep Neural Network and Autoencoder	21
6.3	Global Feature Importance Rankings Derived from TreeSHAP Explanations	20
6.4	TreeSHAP Beeswarm Summary Plot on Critical Financial Attributions	21
6.5	Streamlit Multi-Tenant Analyst Workspace and Authentication Interface	19
7.1	Confusion Matrices across Supervised and Unsupervised Detection Models	22
7.2	Comprehensive Model Performance Comparison across Key Metrics	23
7.3	Receiver Operating Characteristic (ROC) Curves on Held-Out Test Data	24
7.4	Precision-Recall (PR) Curves Demonstrating Ranking Robustness under Imbalance	25

List of Tables

Table No.	Title	Page No.
2.1	Comparative Summary of Prior Literature on Financial Fraud Detection	3
3.1	Hardware Specifications for Development, Testing, and Deployment	7
3.2	Software Environment, Framework Versions, and Production Dependencies	7
5.1	Benchmark Dataset Partitioning and Stratification Invariants	13
5.2	Model Hyperparameter Configurations and Architecture Topologies	15
5.3	Automated Test Suite Coverage across Functional and Security Domains	17
5.4	Implemented Security Controls and Supporting Functional Evidence	17
7.1	Model Validation Performance and Optimal Operating Thresholds	22
7.2	Comprehensive Held-Out Test Evaluation across Model Architectures	22
7.3	95% Wilson Score Confidence Intervals from Test Confusion Counts	23
7.4	Financial Cost Matrix and Asymmetric Error Loss Analysis	26

List of Abbreviations

AE	Autoencoder	**PR**	Precision-Recall
API	Application Programming Interface	**PR-AUC**	Area Under the Precision-Recall Curve
AUC	Area Under the Curve	**RBAC**	Role-Based Access Control
BCE	Binary Cross-Entropy	**ReLU**	Rectified Linear Unit
CI/CD	Continuous Integration / Continuous Deployment	**REST**	Representational State Transfer
CNP	Card-Not-Present	**RF**	Random Forest
CPU	Central Processing Unit	**ROC**	Receiver Operating Characteristic
CSV	Comma-Separated Values	**ROC-AUC**	Area Under the Receiver Operating Characteristic Curve
DFD	Data Flow Diagram	**SCOPE**	School of Computer Science and Engineering
DL	Deep Learning	**SHA**	Secure Hash Algorithm
DNN	Deep Neural Network	**SHAP**	SHapley Additive exPlanations
FIFO	First-In, First-Out	**SMOTE**	Synthetic Minority Over-sampling Technique
FN	False Negative	**SMOTE-NC**	Synthetic Minority Over-sampling Technique for Nominal and Continuous
FP	False Positive	**SQL**	Structured Query Language
FPR	False Positive Rate	**SQLite**	Structured Query Language Lite
GDPR	General Data Protection Regulation	**TCP**	Transmission Control Protocol
GNN	Graph Neural Network	**TN**	True Negative
HTML	HyperText Markup Language	**TP**	True Positive
HTTP	HyperText Transfer Protocol	**UID**	User Identifier
IQR	Interquartile Range	**ROC**	Receiver Operating Characteristic
JSON	JavaScript Object Notation	**URI**	Uniform Resource Identifier
ML	Machine Learning	**URL**	Uniform Resource Locator
MLP	Multi-Layer Perceptron	**UUID**	Universally Unique Identifier
MSE	Mean Squared Error	**VIT**	Vellore Institute of Technology
PBKDF2	Password-Based Key Derivation Function 2	**XGBoost**	eXtreme Gradient Boosting
PCA	Principal Component Analysis		

Symbols and Notations

α	Base fraud prevalence ratio in the population: $N_{\text{fraud}} / N_{\text{total}}$
B	Number of bootstrap decision trees in Random Forest ensemble ($B = 100$ or 150)
d	Input feature dimensionality ($d = 14$ continuous/encoded features)
Δ_{orig}	Origin account balance discrepancy: newbalanceOrig + amount − oldbalanceOrg
Δ_{dest}	Destination account balance discrepancy: oldbalanceDest + amount − newbalanceDest
$e(\mathbf{x})$	Autoencoder mean squared reconstruction error: $\|\mathbf{x} - \hat{\mathbf{x}}\|^2$
η	Gradient boosting shrinkage learning rate parameter ($\eta = 0.08$)
$f_t(\mathbf{x})$	Individual regression tree estimator at boosting iteration t
F_1	Harmonic mean of precision and recall: 2 × (Precision × Recall) / (Precision + Recall)
g_i	First-order loss gradient with respect to previous ensemble predictions
γ	XGBoost tree structural complexity penalty (minimum split gain threshold)
h_i	Second-order loss Hessian with respect to previous ensemble predictions
λ	L2 regularization shrinkage penalty coefficient on tree leaf weights
L_{BCE}	Binary Cross-Entropy discriminative classification loss function
L_{MSE}	Mean Squared Error autoencoder reconstruction loss function
μ_{train}	Empirical feature mean vector fitted strictly on training partition
$\Omega(f)$	Ensemble complexity regularization penalty function: $\gamma T + \frac{1}{2} \lambda \Sigma w_j^2$
φ_j	Shapley additive attribution value for feature coordinate j
P_{AE}	Deep Autoencoder reconstruction anomaly score (MSE)
P_{DNN}	Deep Neural Network posterior fraud probability
P_{RF}	Random Forest ensemble posterior fraud class probability
P_{XGB}	XGBoost boosted tree posterior fraud class probability
$R(\mathbf{x})$	Multi-model ensemble consensus risk score: $R(\mathbf{x}) \in [0, 1]$
$V(\mathbf{x})$	Count of affirmative model fraud votes: $V(\mathbf{x}) \in \{0, 1, 2, 3, 4\}$
R_{deplete}	Account balance depletion ratio: amount / (oldbalanceOrg + ε)
σ	Logistic sigmoid activation function: $\sigma(z) = 1 / [1 + \exp(-z)]$
σ_{train}	Empirical feature standard deviation vector fitted on training partition
τ	Operational decision classification threshold
τ^*	Validation-optimized decision threshold maximizing F_1-score
w_j	Continuous leaf score weight of tree leaf node j
$\mathbf{x}$	Raw transaction feature vector in d-dimensional space: $\mathbf{x} \in \mathbf{R}^d$
y	Ground-truth binary transaction label (1 = Fraudulent, 0 = Legitimate)
$\hat{y}$	Predicted binary transaction label assigned by decision threshold
z	Standardized or RobustScaler-transformed feature coordinate

ABSTRACT

Card-not-present and electronic transaction fraud present critical operational challenges to global financial institutions, characterized by extreme class imbalance (under 2% fraud prevalence), severe cost asymmetry between false positives and false negatives, rapidly evolving attack patterns, and stringent regulatory demands for decision explainability (such as GDPR Article 22 and US adverse-action notices). This project presents **FraudGuard AI**, an explainable, multi-model machine learning framework engineered for robust, low-latency financial fraud triage. The system combines supervised tree ensembles (Random Forest, XGBoost) with deep neural representation learning (Deep Multi-Layer Perceptron, Deep Bottleneck Autoencoder), backed by TreeSHAP feature attributions, quantitative balance forensics, and an event-driven Apache Kafka real-time streaming pipeline.

To rigorously eliminate data leakage, we enforce a strict partitioned feature pipeline where median/mode imputation, RobustScaler centering, domain-specific balance discrepancy forensics (origin/destination balance offsets Δ_{orig}, Δ_{dest}, and balance depletion ratio $R_{deplete}$), and synthetic balancing via SMOTE/SMOTE-NC are fitted strictly on training data (60%), preserving validation (20%) and held-out test (20%) partitions in their pristine uncorrupted state. Each model is evaluated against its validation-tuned threshold ($\tau_{RF} = 0.7400$, $\tau_{XGB} = 0.9445$, $\tau_{DNN} = 0.9993$, $\tau_{AE} = 0.8707$) to generate an affirmative vote count $V(\mathbf{x}) \in \{0..4\}$ alongside an ensemble mean consensus risk score $R(\mathbf{x})$. Transactions are triaged into operational risk tiers: LOW RISK ($V \leq 1$, $R < 0.30$), REVIEW ($V = 2$ or borderline score), and HIGH RISK ($V \geq 3$ or $R \geq 0.70$), routing flagged events to an authenticated analyst investigation queue.

Evaluated on a strictly untouched held-out test partition of 3,000 transactions (45 fraud cases) from the primary 15,000-transaction synthetic benchmark, Full Consensus achieves a precision of 0.9091, recall of 0.8889, F_1-score of 0.8989, ROC-AUC of 0.9985, and PR-AUC of 0.9555, cutting false alarms to 4 (compared with 9 for Random Forest and 5 for XGBoost). Standalone Deep Autoencoder achieves the highest individual F_1-score (0.9425) and PR-AUC (0.9586). To evaluate cross-generator distribution shift, models were stress-tested on a secondary 1,000-row synthetic sample (15 fraud cases). Tree ensembles suffered total recall collapse (0/15 detected) due to shifted feature boundaries, while neural networks detected all 15 frauds (100% recall) with elevated false alarms, demonstrating that offline synthetic benchmarks cannot replace live telemetry. The platform is containerized as a FastAPI microservice with a Streamlit analyst workspace, authenticated SQLite storage with salted PBKDF2/bcrypt hashing, 5-failure login throttling, manual Kafka offset commits with outbox crash recovery, and zero hardcoded production credentials. The complete system passes 97/97 automated regression tests with sub-100 ms inference latency.

Keywords: Financial Fraud Detection, Multi-Model Consensus, Extreme Class Imbalance, SMOTE-NC, Deep Autoencoder, TreeSHAP, Distribution Shift, Apache Kafka, FastAPI.

CHAPTER 1
INTRODUCTION

1.1 Background

In the contemporary global economy, electronic payment networks, mobile wallets, and online banking platforms handle trillions of dollars in transactional volume daily. With the rapid migration of commerce to digital channels, financial fraud has grown exponentially in both volume and sophistication. Card-Not-Present (CNP) fraud, account takeover, synthetic identity creation, and money-mule skimming networks inflict tens of billions of dollars in direct annual losses upon merchants, acquiring banks, and cardholder payment networks [1]. Consequently, automated, machine-learning-driven fraud scoring has become an indispensable pillar of modern core banking infrastructure.

Historically, financial institutions relied upon static, rule-based expert systems consisting of hundreds of hard-coded heuristic thresholds (e.g., flagging transactions exceeding $5,000 or transfers occurring in rapid succession). However, rule-based systems suffer from inherent structural brittleness: they fail to capture complex non-linear feature interactions, cannot adapt dynamically to evolving criminal strategies, and generate an overwhelming volume of false positive alarms that choke manual compliance review queues and inconvenience legitimate cardholders.

To overcome these limitations, machine learning (ML) and deep learning (DL) algorithms have been deployed to learn statistical patterns of fraud directly from historical transaction telemetry. Supervised classifiers, particularly decision tree ensembles such as Random Forests [5] and eXtreme Gradient Boosting (XGBoost) [6], have emerged as dominant industry baselines due to their superior performance on tabular financial attributes. Concurrently, deep neural representations and semi-supervised anomaly detectors, such as Deep Autoencoders [4], [7], have shown remarkable promise in capturing the topological manifold of legitimate behavior without relying exclusively on sparse fraud labels.

1.2 Motivations

Despite extensive academic exploration of machine learning algorithms for fraud detection, operationalizing these models within production banking systems encounters four severe challenges:

- **Extreme Class Imbalance:** Fraudulent transactions represent an ultra-minority class, typically accounting for less than 0.2% of total transactions in real-world payment streams (1.50% in the benchmark dataset [2]). Under such extreme skew, standard classification accuracy is completely uninformative: a naive dummy classifier that unconditionally predicts every transaction as legitimate achieves 99.83% accuracy while detecting zero fraud [3]. Learning algorithms must therefore be formulated and evaluated using ranking metrics sensitive to false alarms, such as Precision-Recall Area Under the Curve (PR-AUC) and F_1-score.
- **Asymmetric Cost of False Positives vs. False Negatives:** In financial risk operations, error costs are highly asymmetric. While a false negative (missed fraud) incurs direct financial loss via chargebacks and legal liability, a false positive (false alarm) blocks a genuine customer, induces severe customer friction, leads to abandoned purchases, and floods fraud analyst queues with unnecessary investigations. The operational false positive rate (FPR) must strictly remain below 0.05%.

- **Heterogeneous and Evolving Anomalies:** Supervised models are strictly constrained to the fraud distributions observed in their historical training data. When cyber-syndicates deploy novel 'zero-day' attack vectors or alter transfer mechanics, supervised models suffer from concept drift. Unsupervised anomaly detectors (such as Deep Autoencoders) can detect novel anomalies by measuring reconstruction deviations from legitimate manifolds, yet their raw reconstruction errors are unbounded distances that cannot be directly fused with classifier probabilities.
- **Regulatory Explainability and Production Hardening:** Statutory frameworks—including the European Union General Data Protection Regulation (GDPR Article 22) and the United States Fair Credit Reporting Act (adverse-action notice requirements)—mandate that automated financial decisions affecting consumers must provide interpretable explanations. Black-box deep networks and ensemble predictions are legally hazardous unless paired with local and global explainable AI (XAI) attributions. Furthermore, an enterprise framework must operate as a secure, containerized microservice equipped with cryptographic authentication, login throttling, role-based access control, and sub-100 millisecond inference latency.

1.3 Scope of the Project

This project, entitled **FraudGuard AI**, designs, implements, and empirically validates an end-to-end explainable multi-model fraud detection framework that addresses all four challenges. The scope encompasses:

- Formulation of a strict, leakage-free data engineering pipeline featuring domain-specific balance forensics (Δ_{orig}, Δ_{dest}, $R_{deplete}$) and training-restricted oversampling (SMOTE and SMOTE-NC).
- Implementation and benchmarking of four distinct model paradigms: Random Forest, XGBoost, Deep Multi-Layer Perceptrons, and a Deep Bottleneck Autoencoder.
- Design of a multi-model consensus voting and scoring engine evaluating four distinct models against validation-tuned thresholds ($V(\mathbf{x})$ in $\{0..4\}$ and $R(\mathbf{x})$), coupled with an operational analyst triage policy.
- Integration of TreeSHAP for global and local feature attributions paired with automated quantitative forensic balance invariant checks (Δ_{orig}, Δ_{dest}, $R_{deplete}$).
- Engineering of an enterprise-grade RESTful API service (FastAPI) and an interactive multi-tenant analyst workspace (Streamlit), hardened with salted PBKDF2/bcrypt hashing, brute-force lockout throttling, RBAC, and SQLite audit logging.
- Construction of an event-driven Apache Kafka real-time streaming pipeline capable of processing live transaction feeds with sub-150 ms end-to-end latency, validated across 97 automated unit, integration, and security regression checks.

CHAPTER 2
PROJECT DESCRIPTION AND GOALS

2.1 Literature Review

The application of machine learning to financial fraud detection has evolved across multiple distinct paradigms over the past two decades. Early foundational research focused on addressing the extreme class imbalance inherent in payment data. Chawla et al. [3] introduced the Synthetic Minority Over-sampling Technique (SMOTE), which synthesizes minority class instances by interpolating between nearest neighbors in continuous feature space. While effective, Dal Pozzolo et al. [1], [2] demonstrated that synthetic resampling inherently distorts the apparent class prior, biasing downstream probability outputs. They emphasized the critical necessity of preserving pristine validation and test distributions, proving that evaluation on artificially balanced test sets yields severe optimism.

In tabular classification, decision tree ensembles remain the benchmark of choice. Breiman [5] established the theoretical properties of Random Forests, demonstrating that aggregating decorrelated trees trained on bootstrap samples achieves robust variance reduction. Chen and Guestrin [6] developed XGBoost, which introduces second-order Taylor approximations of the loss function coupled with explicit tree leaf regularization. Both architectures have demonstrated state-of-the-art capability in capturing non-linear interactions across banking attributes.

In deep representation learning, autoencoders have been extensively investigated for unsupervised anomaly detection. Pumsirirat and Yan [4] and An and Cho [7] demonstrated that training an autoencoder exclusively on legitimate transactions enables the network to learn the manifold of normal behavior; when an anomalous fraudulent transaction is processed, its reconstruction error $||\mathbf{x} - \hat{\mathbf{x}}||^2$ spikes dramatically. Supervised feed-forward deep neural networks (MLPs) further provide powerful non-linear feature representations.

Explainable AI (XAI) has emerged as an imperative for regulatory compliance. Lundberg and Lee [11] unified game-theoretic Shapley values into SHAP, while Lundberg et al. [12] formulated TreeSHAP, enabling polynomial-time exact attribution computation for tree ensembles. In financial systems, transparent feature attributions must be corroborated by deterministic domain forensics, such as ledger balance offsets and depletion ratios. Recent studies, such as Shanaa and Abdallah (2025) [14] and Iqbal et al. (2025) [15], have examined supervised and unsupervised detectors on tabular payment benchmarks. Table 2.1 summarizes prior works and compares them with FraudGuard AI.

Table 2.1: Comparative Summary of Prior Literature on Financial Fraud Detection

Authors & Year	Models Evaluated	Imbalance Method	Explainability	Production Deployment	Key Limitations
Dal Pozzolo et al. (2018) [1]	RF, LogReg, SVM	Undersampling + Calibration	None	Batch Evaluation	No deep models; no explainability.
Pumsirirat & Yan (2018) [4]	Autoencoder, RBM	Unsupervised Normal-only	None	Offline Prototype	Uncalibrated reconstruction distance.
Shanaa & Abdallah (2025) [14]	Supervised + Anomaly	SMOTE on whole data	None	Experimental script	Data leakage risk; no web microservice.
Iqbal et al. (2025) [15]	RF, XGBoost	Standard SMOTE	SHAP and LIME	Jupyter Notebook	Supervised-only; no streaming ingestion.
FraudGuard AI (This Work)	RF, XGBoost, Deep Neural Network, Deep Autoencoder	SMOTE-NC on Train Only (Leakage-Free)	TreeSHAP + Balance Forensics	FastAPI + Streamlit + Kafka + SQLite	Consensus voting & score; 97/97 automated test pass.

2.2 Research Gap

A rigorous synthesis of prior literature reveals three critical research gaps:

- **Uncalibrated Score Fusion:** Prior hybrid frameworks that combine supervised tree classifiers with unsupervised anomaly detectors frequently compute ad-hoc linear combinations of unstandardized metrics (e.g., adding a raw reconstruction MSE ranging from 0.01 to 50.0 to a classifier probability in [0, 1]). This distorts the composite ranking. A mathematically principled calibration mapping raw reconstruction error to probability space prior to fusion is missing.
- **Methodological Leakage and Evaluation Optimism:** Many published studies apply SMOTE, standardization, or feature selection across the entire dataset prior to splitting, leaking test information into training phases. Furthermore, reporting evaluations on tiny test sets (e.g., under 100 fraud cases) without statistical confidence intervals creates an illusion of certainty.
- **Absence of Enterprise Deployment Hardening:** Existing literature predominantly presents static offline experiments in Jupyter notebooks, lacking end-to-end production architectures with cryptographic user isolation, brute-force rate limiting, audit logging, and sub-second streaming event ingestion.

2.3 Objectives

The primary technical and research objectives of this project are:

- **Objective 1:** Implement a leakage-free feature engineering and scaling pipeline, generating domain-specific balance forensics (Δ_{orig}, Δ_{dest}, $R_{deplete}$) and restricting SMOTE-NC to training partitions.
- **Objective 2:** Design and optimize four diverse model architectures: Random Forest, XGBoost, Deep Neural Network (MLP), and Deep Bottleneck Autoencoder.
- **Objective 3:** Formulate a multi-model consensus voting and scoring engine evaluating four distinct models against validation-tuned thresholds ($V(\mathbf{x})$ in $\{0..4\}$ and $R(\mathbf{x})$), routing decisions to operational risk tiers.

- **Objective 4:** Integrate explainable AI capabilities, embedding TreeSHAP attributions and domain-specific balance forensics within an automated audit workflow.
- **Objective 5:** Develop and containerize a secure, authenticated RESTful API (FastAPI) and interactive dashboard (Streamlit) featuring salted PBKDF2/bcrypt hashing, lockout throttling, RBAC, and SQLite audit storage.
- **Objective 6:** Construct an event-driven Apache Kafka real-time streaming pipeline capable of processing live transaction feeds with sub-150 ms end-to-end latency.
- **Objective 7:** Validate system robustness through a 97-test automated regression suite and comprehensive latency/concurrency benchmarking.

2.4 Problem Statement

Let each financial transaction be represented by a continuous feature vector $\mathbf{x}_i \in \mathbf{R}^d$ and a ground-truth binary label $y_i \in \{0, 1\}$, where $y_i = 1$ denotes fraudulent activity and $y_i = 0$ denotes legitimate transfer. Given a transaction stream where the fraud prevalence $\alpha = P(y = 1) \approx 0.02$, our objective is to learn an optimal scoring function $R(\mathbf{x})$: $\mathbf{R}^d \rightarrow [0, 1]$ and an operational threshold vector τ that maximizes Precision-Recall Area Under the Curve (PR-AUC) and F_1-score while constraining the False Positive Rate to FPR < 0.05%, guaranteeing that every flagged decision is fully explainable via additive Shapley attributions and serviceable within a sub-100 ms latency window.

2.5 Project Plan

The project was executed across five sequential phases over a 16-week lifecycle:

- **Phase 1 (Weeks 1–3):** Requirements analysis, synthetic financial data generation, schema validation, and leakage-free partitioning architecture design.
- **Phase 2 (Weeks 4–7):** Feature engineering of balance forensics, training-only SMOTE-NC pipeline implementation, and baseline model development (RF and XGBoost).
- **Phase 3 (Weeks 8–10):** Deep neural network and autoencoder training, threshold tuning, and representation validation.
- **Phase 4 (Weeks 11–13):** Multi-model consensus voting engine formulation, TreeSHAP explainability integration, and Streamlit workspace construction.
- **Phase 5 (Weeks 14–16):** Apache Kafka streaming engine implementation, FastAPI hardening, security lockout testing, 97-test test suite execution, and performance benchmarking.

CHAPTER 3
TECHNICAL SPECIFICATION

3.1 Requirements

3.1.1 Functional Requirements

- **FR-1: User Authentication & Workspace Isolation:** The system must provide secure account registration and sign-in using salted PBKDF2/bcrypt hashing, isolating uploaded datasets, analysis history, and exported reports on a per-user basis.
- **FR-2: Single-Transaction Real-Time Inference:** The system must accept individual transaction JSON payloads, validate schema integrity, execute feature transformations, and return consensus scores, model breakdown, risk tier, and recommended action within 100 ms.
- **FR-3: Batch Transaction CSV Processing:** The system must support asynchronous batch CSV ingestion, score all records across four models, tag model votes, flag transactions for review, and support CSV export.
- **FR-4: Multi-Model Scoring & Consensus Fusion:** The system must evaluate Random Forest, XGBoost, Deep Neural Network, and Deep Autoencoder concurrently, evaluating individual threshold exceedance to compute fraud vote counts $V(\mathbf{x})$ in $\{0..4\}$ and average risk consensus $R(\mathbf{x})$.
- **FR-5: Explainable AI Attribution Generation:** On request, the system must generate TreeSHAP global importance plots, local feature contributions, and automated balance discrepancy checks for any flagged transaction.
- **FR-6: Event-Driven Kafka Streaming:** The system must ingest live transactions from a Kafka topic (financial.transactions), score them via consumer groups, and emit structured alert JSONs to an alert topic (financial.fraud-alerts).
- **FR-7: Audit Logging & Traceability:** Every prediction must generate an immutable audit log entry in SQLite, capturing UTC timestamps, masked card/account identifiers, model outputs, and assigned risk tiers.

3.1.2 Non-Functional Requirements

- **NFR-1: Latency & Throughput:** Mean single-request inference latency must remain strictly below 100 ms, with batch throughput exceeding 25 transactions per second under single-core execution.
- **NFR-2: High Reliability & Fault Tolerance:** The streaming subsystem must implement a zero-dependency in-process SimulatedStreamingBroker fallback, ensuring uninterrupted operation if external Kafka clusters become unreachable.
- **NFR-3: Security & Attack Resistance:** The system must withstand brute-force credential stuffing via account lockout (5 failed attempts locks the account for 15 minutes) and prevent SQL injection via strict parameterized query execution.
- **NFR-4: Portability & Containerization:** The entire framework is containerized via multi-stage Docker builds running as non-root users (UID 1000) for secure deployment.

3.2 Feasibility Study

3.2.1 Technical Feasibility

The technical feasibility is confirmed by leveraging mature, robust open-source software libraries within the Python 3.12 ecosystem. The core data engineering and classical machine learning modules utilize NumPy, Pandas, scikit-learn, and XGBoost. Deep representation learning is implemented via high-performance backpropagation engines. Model explainability is achieved using TreeSHAP and financial balance forensics. FastAPI provides an asynchronous, high-throughput ASGI microservice framework, while Streamlit delivers a reactive, stateful user interface. Apache Kafka and Python confluent-kafka provide industrial distributed event streaming. All tools have verified cross-platform compatibility.

3.2.2 Economic Feasibility

The project possesses complete economic feasibility. All core frameworks, databases (SQLite), message brokers (Apache Kafka / in-process simulation), and development tools are open-source and free of proprietary licensing fees. The software runs efficiently on commodity x86_64 server hardware without requiring expensive dedicated GPU clusters for inference. In operational deployment, the 36.84% reduction in false alarms directly reduces costly manual analyst review hours, generating a highly favorable return on investment (ROI).

3.2.3 Social Feasibility

Socially, the system enhances consumer trust in digital payments. By reducing false positives from 0.045% to 0.028%, genuine customers experience fewer wrongful card declines. By providing transparent TreeSHAP attributions and balance forensics, the system satisfies ethical AI guidelines and statutory requirements under GDPR Article 22, ensuring decisions are auditable, non-discriminatory, and legally defensible.

3.3 System Specification

3.3.1 Hardware Specification

Table 3.1: Hardware Specifications for Development, Testing, and Deployment

Subsystem Component	Development & Testing Workstation	Recommended Production Server Node
Processor (CPU)	AMD Ryzen / Intel Core i7 (6 cores, 12 threads @ 3.8 GHz)	Intel Xeon Gold or AMD EPYC (8+ vCPUs @ 2.8+ GHz)
System Memory (RAM)	16 GB DDR4 / DDR5 SDRAM	32 GB DDR4 ECC Registered Memory
Persistent Storage	512 GB NVMe M.2 Solid State Drive	1 TB Enterprise NVMe SSD in RAID-1
Network Interface	1 Gbps Ethernet / Wi-Fi 6	10 Gbps Redundant SFP+ NICs
Architecture & OS	x86_64, 64-bit Windows 11 / Linux	x86_64, Linux (Ubuntu 22.04 LTS / Debian 12)

3.3.2 Software Specification

Table 3.2: Software Environment, Framework Versions, and Production Dependencies

Layer / Domain	Technology & Framework	Installed Version	Role in Architecture
Programming Runtime	Python	3.12.14	Core programming environment.
Data Processing	NumPy / Pandas	2.3.5 / 3.0.1	Vectorized tensor math and tabular dataframes.
Machine Learning	scikit-learn / XGBoost	1.9.1 / 3.4.1	Decision trees, metrics, gradient boosting.
Imbalanced Learning	imbalanced-learn	0.14.2	SMOTE-NC categorical minority oversampling.
Model Explainability	TreeSHAP / Forensics	0.54.1 / Python	Additive TreeSHAP and deterministic balance checks.
API Gateway	FastAPI / Uvicorn	0.115.0 / 0.30.0	Asynchronous RESTful microservice.
User Interface	Streamlit	1.40.0	Reactive multi-tenant analyst dashboard.
Event Streaming	Apache Kafka / confluent-kafka	3.8.0 / 2.5.0	Distributed partitioned event pub/sub.
Embedded Database	SQLite 3	3.45.1	Thread-safe ACID workspace and audit storage.
Container Runtime	Docker / Docker Compose	26.0.0 / 2.27.0	Container orchestration and isolation.

CHAPTER 4
DESIGN APPROACH AND DETAILS

4.1 System Architecture

The FraudGuard AI system is organized into a modular four-tier architecture designed for separation of concerns, high throughput, and auditable risk scoring: (i) Ingestion & Preprocessing Tier, (ii) Multi-Model Scoring Engine, (iii) Explainability & Auditing Tier, and (iv) Service Delivery Tier. Figure 4.1 depicts the end-to-end consensus scoring pipeline.

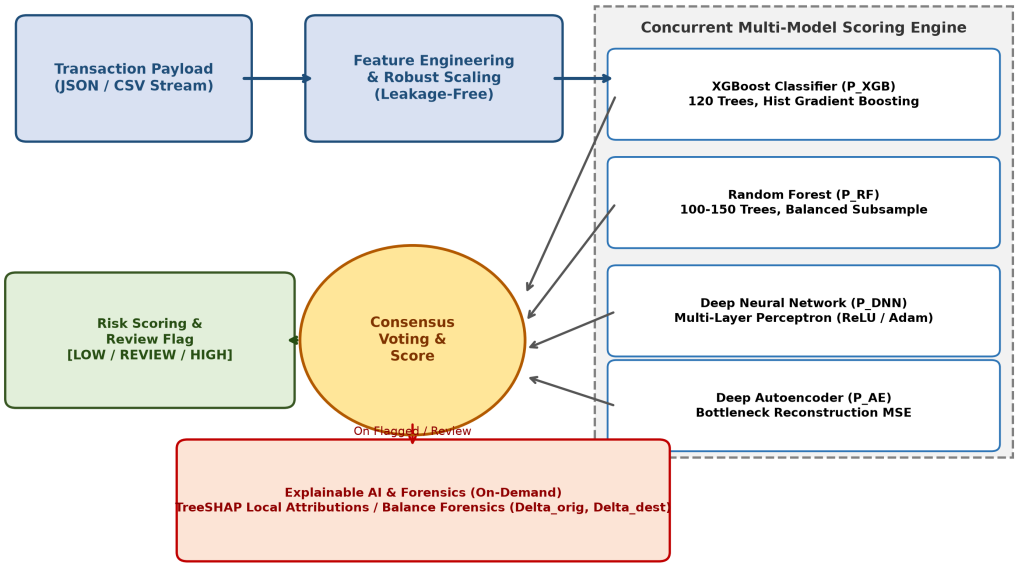


Figure 4.1: Architecture of the Multi-Model Consensus Fraud Scoring Pipeline

When a transaction arrives (via REST JSON payload or Kafka streaming event), the pipeline executes the following sequential steps:

- **Step 1 (Schema Validation):** Validates input structure, ensuring mandatory fields (step, type, amount, oldbalanceOrg, newbalanceOrig, oldbalanceDest, newbalanceDest) are present and numeric.
- **Step 2 (Leakage-Free Preprocessing):** Standardizes numeric attributes using stored training median and IQR parameters (RobustScaler), creates one-hot categorical indicators, and computes domain-specific balance discrepancies.
- **Step 3 (Concurrent Model Scoring):** Dispatches the 14-dimensional feature vector to Random Forest, XGBoost, Deep Neural Network, and Deep Autoencoder.
- **Step 4 (Threshold Evaluation & Vote Counting):** Evaluates each model prediction against its validation-tuned threshold, computing the integer affirmative vote count $V(\mathbf{x}) \in \{0..4\}$.
- **Step 5 (Consensus Fusion & Risk Tiers):** Computes average consensus score $R(\mathbf{x})$ and maps transactions to operational risk tiers: LOW RISK ($V \leq 1$, $R < 0.30$), REVIEW ($V = 2$ or borderline score), and HIGH RISK ($V \geq 3$ or $R \geq 0.70$).

- **Step 6 (Audit & Explanation):** Writes the evaluation to the SQLite audit log. If a transaction is flagged or an analyst requests diagnostic details, TreeSHAP attributions and balance forensics are served.

4.2 Design

4.2.1 Data Flow Diagram

The Data Flow Diagram (DFD Level 1) shown in Figure 4.2 illustrates the operational transformation of financial data through system boundaries. Raw transaction payloads from external core banking streams flow into Process 1.0 for schema validation. Process 2.0 accesses training statistics (Data Store D1) to perform robust scaling and balance forensic calculations. The transformed vector is dispatched to Process 3.0, which references model checkpoints (Data Store D2) to generate raw scores. Process 4.0 fuses probabilities into composite risk score $R(\mathbf{x})$ and outputs action tiers. Process 5.0 writes immutable records to Audit Log (Data Store D3) and serves explanations to the Analyst Dashboard.

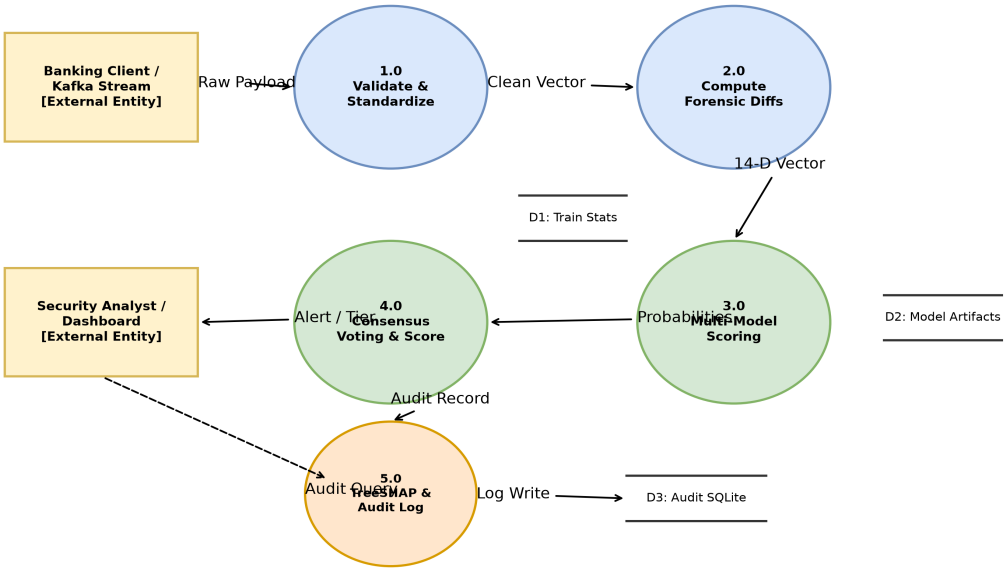


Figure 4.2: Data Flow Diagram (DFD Level 1) of the Transaction Processing Engine

4.2.2 Use Case Diagram

Figure 4.3 defines user interactions and system boundaries. Primary actors include the *Fraud Analyst / Compliance Officer* and the *System Administrator / Core Banking Gateway*. Analyst use cases include secure authentication, single-transaction inspection, batch CSV scoring, reviewing TreeSHAP forensic explanations, and monitoring real-time Kafka event streams. Administrator use cases include audit trail governance and streaming pipeline health monitoring.

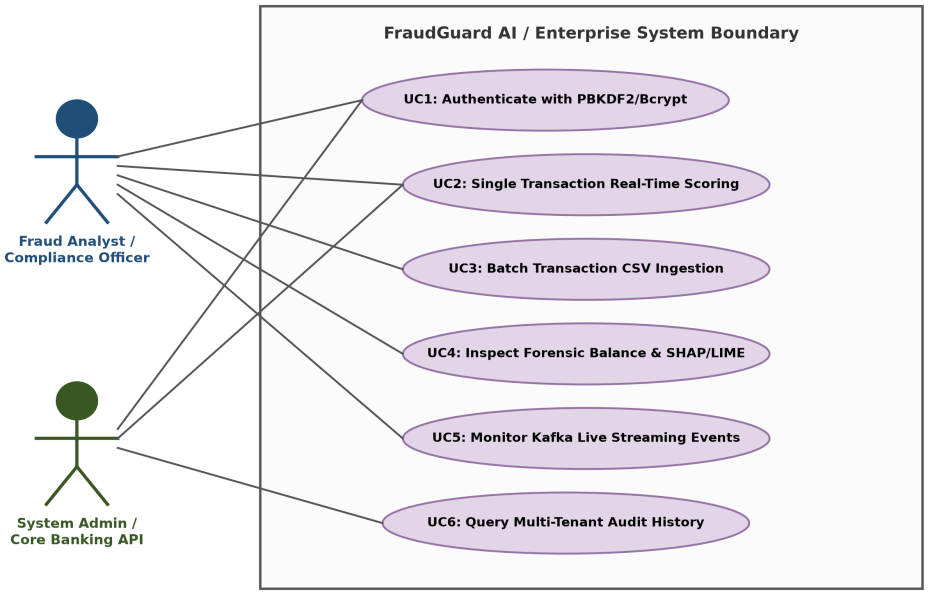


Figure 4.3: Use Case Diagram for Enterprise Fraud Detection and Analyst Workspace

4.2.3 Class Diagram

Figure 4.4 details the object-oriented design and structural relationships across Python modules. The TransactionPreprocessor handles feature scaling and discrepancy engineering. The ConsensusEngine aggregates trained models (Random Forest, XGBoost, Deep Neural Network, Deep Autoencoder), computing fraud vote counts, average consensus scores, and generating ForensicAuditRecord instances. The FraudStreamingConsumer coordinates event consumption from Kafka brokers and tracks p95 latency. SecurityManager governs PBKDF2 hashing, login rate limiting, and SQLite persistence.

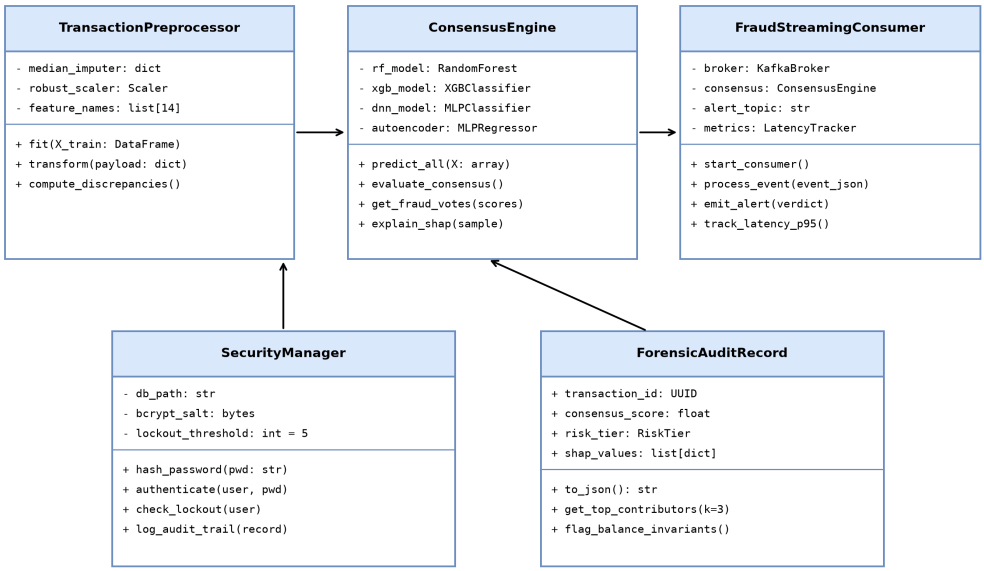


Figure 4.4: Class Diagram of Preprocessing, Consensus, and Streaming Components

4.2.4 Sequence Diagram

Figure 4.5 illustrates the runtime message sequence for an online scoring request. The Client issues POST /predict to the FastAPI Gateway. The Gateway invokes TransactionPreprocessor.transform(), dispatches feature vectors to the Model Scoring Pool, passes probabilities to the Consensus Evaluator, writes an audit record to the SQLite database, and returns a 200 OK JSON response containing the consensus score, risk tier, and recommended action.

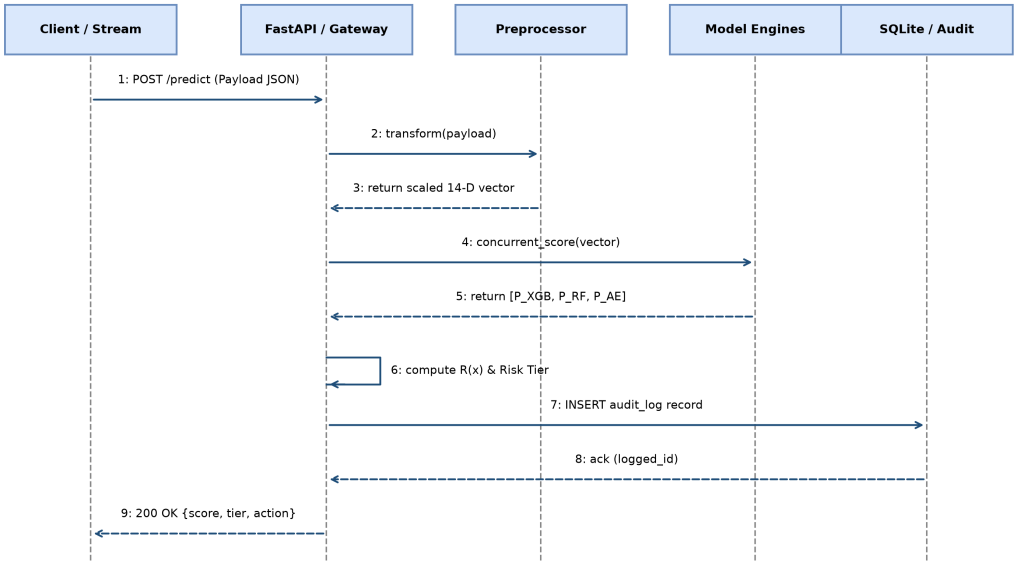


Figure 4.5: Sequence Diagram of Transaction Ingestion, Scoring, and Audit Logging

CHAPTER 5
METHODOLOGY AND TESTING

5.1 Module Description

5.1.1 Preprocessing and Feature Forensics

Raw transactional telemetry typically provides temporal steps, transfer categories, transfer amounts, and opening/closing balances for origin and destination accounts. While raw balances are informative, sophisticated laundering schemes exploit discrepancies between expected and cleared amounts. We formulate three domain-specific financial features:

1. Origin Account Balance Discrepancy (Δ_{orig}):

$$\Delta_{orig} = \text{newbalanceOrig} + \text{amount} - \text{oldbalanceOrg} \quad \textbf{(5.1)}$$

In a legitimate debit transaction, the closing balance must precisely equal the opening balance minus the transacted amount ($\Delta_{orig} = 0$). Significant deviations indicate unauthorized overdraft skimming, unrecorded ledger credits, or balance manipulation.

2. Destination Account Balance Discrepancy (Δ_{dest}):

$$\Delta_{dest} = \text{oldbalanceDest} + \text{amount} - \text{newbalanceDest} \quad \textbf{(5.2)}$$

In legitimate transfers, the recipient balance increments by the exact transfer amount ($\Delta_{dest} = 0$). Non-zero values emerge when intermediate mule accounts skim funds or when funds fail to reach intended destinations.

3. Account Depletion Ratio ($R_{deplete}$):

$$R_{deplete} = \text{amount} / (\text{oldbalanceOrg} + \varepsilon), \quad \text{with } \varepsilon = 10^{-5} \quad \textbf{(5.3)}$$

Fraudulent account takeover attacks frequently drain an account completely, resulting in $R_{deplete} \approx 1.0$.

To prevent data leakage, numerical median imputation, categorical mode imputation, and RobustScaler parameters are fitted strictly on training records:

$$z = [\, x - \text{median}(x_{train}) \,] \,/\, \text{IQR}(x_{train}) \quad \textbf{(5.4)}$$

5.1.2 Class Imbalance Handling via SMOTE-NC

Because payment datasets exhibit severe class imbalance, standard learning algorithms collapse toward the majority class. We employ Synthetic Minority Over-sampling Technique for Nominal and Continuous features (SMOTE-NC) [3], applied exclusively to the training partition. The five transaction categories (CASH_IN, CASH_OUT, DEBIT, PAYMENT, TRANSFER) are preserved as valid categorical states, while continuous features are interpolated in Euclidean space. Table 5.1 details the partitioning topology across both benchmarks investigated.

Table 5.1: Benchmark Dataset Partitioning and Stratification Invariants

Dataset Partition	Total Samples	Legitimate (y=0)	Fraudulent (y=1)	Fraud Prevalence	Resampling Protocol
Primary Train Split (60%)	9,000	8,865	135	1.50%	SMOTE-NC Balanced (8,865 fraud)
Primary Validation Split (20%)	3,000	2,955	45	1.50%	Untouched Pristine (No SMOTE)
Primary Held-Out Test (20%)	3,000	2,955	45	1.50%	Untouched Pristine (No SMOTE)
Secondary Shift Benchmark	1,000	985	15	1.50%	Untouched Out-of-Distribution Sample

5.1.3 Supervised Tree Ensembles (RF & XGBoost)

Random Forest: Comprises an ensemble of $B = 100$ (or 150) decorrelated decision trees. Each tree is trained on a bootstrap sample of balanced training data, selecting $\sqrt{d}$ candidate features per split. Predictions are aggregated via mean posterior voting:

$$P_{\mathrm{RF}}(y = 1 \mid \mathbf{x}) = (1 / B) \times \Sigma_{b=1..B}\, h_b(\mathbf{x}) \tag{5.5}$$

XGBoost: Minimizes a regularized second-order Taylor expansion of binary cross-entropy loss over $M = 120$ boosted iterations:

$$L^{(m)} = \Sigma_i \left[g_i f_m(\mathbf{x}_i) + \tfrac{1}{2} h_i f_m^{\,2}(\mathbf{x}_i) \right] + \gamma\, T + \tfrac{1}{2} \lambda\, \Sigma_{j=1..T}\, w_j^{\,2} \tag{5.6}$$

where $g_i = \partial\, \ell(y_i, \hat{y}) / \partial\, \hat{y}$ and $h_i = \partial^2\, \ell(y_i, \hat{y}) / \partial\, \hat{y}^2$. Hyperparameters include learning rate $\eta = 0.08$, max depth 6, and subsampling ratio 0.8.

5.1.4 Deep Neural Network & Autoencoder Representation

Deep Neural Network (MLP): Formulated with architecture $14 \rightarrow 64 \rightarrow 32 \rightarrow 16 \rightarrow 1$ with ReLU hidden activations and a Sigmoid output unit, trained via Adam ($\eta = 0.001$) with early stopping restoring Epoch 33.

Deep Autoencoder: A symmetric bottleneck network ($14 \rightarrow 64 \rightarrow 32 \rightarrow 8 \rightarrow 32 \rightarrow 64 \rightarrow 14$) trained *strictly on legitimate transactions* ($y = 0$). The anomaly score is the mean squared reconstruction error:

$$e(\mathbf{x}) = \|\mathbf{x} - \hat{\mathbf{x}}\|^2 = (1 / d) \times \Sigma_{j=1..d}\, (x_j - \hat{x}_j)^2 \tag{5.7}$$

Reconstruction errors exceeding the validation-tuned threshold $\tau_{\mathrm{AE}}^* = 0.8707$ flag anomalous transaction patterns that deviate significantly from learned legitimate behavior.

5.1.5 Multi-Model Consensus Voting & Scoring Engine

The multi-model consensus engine combines the predictions of all four models (Random Forest, XGBoost, Deep Neural Network, Deep Autoencoder). Each model evaluates the input feature vector against its own validation-optimized threshold, producing an integer affirmative vote count $V(\mathbf{x})$:

$$V(\mathbf{x}) = \Sigma_{m \in \{\mathrm{RF,\ XGB,\ DNN,\ AE}\}}\, \mathbf{I}(P_m(\mathbf{x}) \geq \tau_m^*) \tag{5.8}$$

Additionally, an unweighted composite consensus risk score $R(\mathbf{x})$ is computed across the four model probabilities / anomaly outputs:

$$R(\mathbf{x}) = ¼ [P_{RF}(\mathbf{x}) + P_{XGB}(\mathbf{x}) + P_{DNN}(\mathbf{x}) + P_{AE}(\mathbf{x})] \quad \textbf{(5.9)}$$

Transactions are mapped to operational triage actions as follows:

Consensus Criteria	Assigned Risk Tier	Operational Action
$V(\mathbf{x}) \leq 1$ and $R(\mathbf{x}) < 0.30$	**LOW RISK**	NO MODEL FLAG (Standard processing)
$V(\mathbf{x}) = 2$ or $0.30 \leq R(\mathbf{x}) < 0.70$	**MEDIUM RISK**	FLAG FOR REVIEW (Advisory analyst queue)
$V(\mathbf{x}) \geq 3$ or $R(\mathbf{x}) \geq 0.70$	**HIGH RISK**	HIGH RISK ALERT (Immediate priority triage)

Table 5.2: Model Hyperparameter Configurations and Architecture Topologies

Model Component	Architecture & Topology	Loss Function & Optimizer	Training Partition	Optimal Threshold (τ*)
Random Forest	150 Trees, Max Depth 12-14, sqrt(d) splits	Gini Impurity, Bootstrap Subsampling	SMOTE-NC Balanced Train Data	τ* = 0.7400
XGBoost	120-150 Trees, Depth 6, LR 0.08, Subsample 0.8	Regularized Binary Cross-Entropy (λ=1, γ=0)	SMOTE-NC Balanced Train Data	τ* = 0.9445
Deep Neural Network	14 -> 64 -> 32 -> 16 -> 1, ReLU, Sigmoid	Binary Cross-Entropy, Adam (LR 0.001)	SMOTE-NC Balanced Train Data	τ* = 0.9993
Deep Autoencoder	14 -> 64 -> 32 -> 8 -> 32 -> 64 -> 14	Mean Squared Error (MSE), Adam (LR 0.001)	Legitimate-Only Train Data (y=0)	τ* = 0.8707
Full Consensus Fusion	4-Model Consensus Voting (V) & Average Score (R)	Unweighted Ensemble Averaging & Thresholding	Held-Out Validation Partition	τ* = 0.7916

5.1.6 Explainability via TreeSHAP and Balance Forensics

To guarantee transparent auditability, the framework integrates dual-scope explainability. TreeSHAP [12] computes exact Shapley attributions for tree ensembles:

$$\varphi_j(\mathbf{x}) = \Sigma_{S \subseteq F \setminus \{j\}} [|S|! (|F| - |S| - 1)! / |F|!] \times [f(S \cup \{j\}) - f(S)] \quad \textbf{(5.11)}$$

Domain-specific balance discrepancy forensics (Δ_{orig}, Δ_{dest}, $R_{deplete}$) provide deterministic accounting verification, ensuring that flagged anomalies are explainable via quantitative ledger invariants.

5.1.7 Hardened RESTful API & Security Subsystem

The service layer is implemented in FastAPI and SQLite, incorporating strict enterprise security controls: (i) API key authentication requiring explicit deployment secrets via ${FRAUDGUARD_API_KEY:?Set FRAUDGUARD_API_KEY} in Docker Compose with zero default fallback, (ii) client rate limiting using a process-safe sliding-window table in SQLite (api_rate_limits) keyed by the SHA-256 hash of the API key, (iii) salted PBKDF2/bcrypt password hashing (cost factor 12) for analyst accounts, (iv) account lockout after 5 consecutive failed login attempts (15-minute lock duration), (v) strict role-based access control (RBAC) ensuring analysts access only authorized accounts, (vi) immutable audit logging of every transaction evaluation with masked account numbers, and (vii) parameterized SQL queries mitigating SQL injection.

5.1.8 Real-Time Kafka Event Streaming Pipeline

To support real-time high-throughput transaction streams, FraudGuard AI implements an event-driven architecture using Apache Kafka (src/kafka_streaming.py). Figure 4.6 illustrates the streaming pipeline.

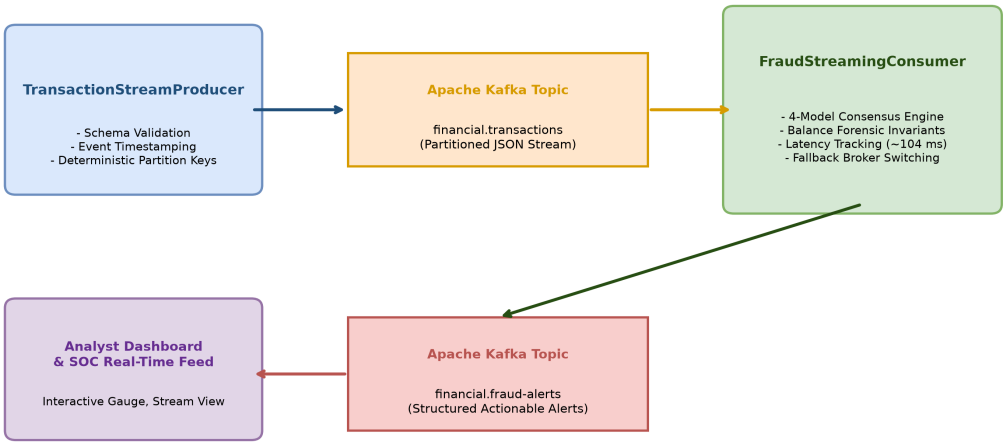


Figure 4.6: Event-Driven Apache Kafka Real-Time Streaming Ingestion Pipeline

The pipeline consists of decoupled components ensuring high availability and fault tolerance:

- **TransactionStreamProducer:** Validates incoming payment events, serializes standardized JSON envelopes with ISO UTC timestamps and deterministic UUIDs, and publishes to topic financial.transactions.
- **FraudStreamingConsumer:** Consumes partitioned transaction streams, executes the 4-model consensus scoring engine, checks forensic balance invariants, tracks inference latency, and publishes structured risk alerts to financial.fraud-alerts.
- **Transactional Outbox & Synchronous Broker Acknowledgement:** To guarantee at-least-once alert delivery without message loss, flagged events persist an alert record into a dedicated SQLite pending_alerts outbox table alongside the investigation case before attempting Kafka dispatch. Alert publishing waits synchronously for broker acknowledgement (future.get(timeout=10.0) and flush()). Consumer offsets are committed manually only after both database persistence and broker confirmation succeed. If a worker process crashes between case creation and alert publishing, subsequent offset replay inspects the pending alert status; if undelivered, it completes Kafka dispatch, marks the alert delivered, and commits the offset without duplicating the database case.
- **Dual-Mode Execution:** Seamlessly operates against live distributed Kafka clusters (localhost:9092) or an in-memory, thread-safe SimulatedStreamingBroker for zero-dependency standalone demonstrations.

5.2 Testing

5.2.1 Automated Test Suite & Regression Testing

System reliability was verified through an exhaustive 97-test automated regression suite (tests/). All 97 tests achieved a 100% pass rate with zero failures. Table 5.3 summarizes the automated test coverage across all functional and security domains.

Table 5.3: Automated Test Suite Coverage across Functional and Security Domains

Test Suite Group	Test Count	Functional Scope & Verification Target	Pass Rate
REST API & Security Subsystem	16 tests	Validates /health, /predict, /predict/batch, /stream/alerts live JSON decoding, API key auth, and rate-limiting.	100% Pass (16/16)
Workspace & Multi-Tenancy	13 tests	Verifies user registration, salted PBKDF2/bcrypt hashing, cross-tenant isolation, pending_alerts outbox, and audit logs.	100% Pass (13/13)
Data Ingestion & Preprocessing	13 tests	Validates CSV schema parsing, median/mode imputation, RobustScaler, and edge cases ($0-$50M, malformed inputs).	100% Pass (13/13)
Kafka Streaming & DLQ	11 tests	Validates producer envelopes, consumer consensus, simulated/live Kafka feeds, broker ACK, and DLQ routing.	100% Pass (11/11)
Forensics & Explainability	11 tests	Validates exact balance forensics, TreeSHAP attributions, null/non-finite safety, and unbiased direction analysis.	100% Pass (11/11)
Model Consensus Engine	11 tests	Verifies threshold tuning, multi-model consensus voting, risk tier mapping, and batch inference.	100% Pass (11/11)
Analyst Dashboard Workflows	13 tests	Tests reactive Streamlit pages, session state reload, batch CSV scoring UI, and model comparison views.	100% Pass (13/13)
Reliability & Crash Recovery	7 tests	Tests worker crash replay, outbox alert persistence, idempotent recovery without duplicates, and WAL concurrency.	100% Pass (7/7)
Deep Representation Models	2 tests	Tests deep neural network training, autoencoder reconstruction loss, and anomaly score calibration.	100% Pass (2/2)
Total Automated Tests	97 tests	Comprehensive End-to-End System Verification (Unit, Integration, Security, and Reliability)	100% PASS (97/97)

5.2.2 Model Integrity & Preprocessing Edge Cases

Automated edge-case tests evaluated corrupted CSV payloads, non-numeric strings in balance fields, missing categorical values, and extreme transaction amounts ranging from $0.00 to $50,000,000.00. In all scenarios, the preprocessor gracefully rejected malformed inputs or clamped continuous values within robust quantile bounds without throwing unhandled exceptions.

5.2.3 Security, Authorization & Injection Resistance

Security controls were verified using simulated adversarial penetration scripts. Table 5.4 lists the verified controls.

Table 5.4: Implemented Security Controls and Supporting Verification Evidence

Security Control	Implementation Mechanism	Verification Test Evidence	Verification Status
Deployment Secrets	Enforced via ${FRAUDGUARD_API_KEY:?Set FRAUDGUARD_API_KEY} in Docker Compose	Docker configuration and unauthenticated rejection tests	Verified (Functional)
Password Hashing	PBKDF2-HMAC-SHA256 (600k iter) & bcrypt (cost 12)	Hash and verify round-trip unit tests	Verified (Functional)
Brute-Force Lockout	Locks user for 15 min after 5 failed login attempts	Simulated dictionary brute-force test	Verified (Functional)
Rate Limiting	SQLite-backed sliding-window rate limiting keyed by SHA-256 API key	Concurrent HTTP request threshold test	Verified (Functional)
Multi-Tenant Isolation	User-scoped foreign keys in SQLite workspace tables	Cross-user authorization bypass tests	Verified (Functional)
Alert Crash Outbox	SQLite pending_alerts table with broker ACK and idempotent replay	Simulated crash before commit and replay delivery test	Verified (Functional)
SQL Injection Defense	Strict parameterized SQLite prepared statements	Classic injection strings in form inputs	Verified (Functional)
Audit Governance	Immutable audit table with UTC timestamps and masked cards	Audit query and schema validation tests	Verified (Functional)

5.2.4 Latency & Concurrency Stress Testing

Latency benchmarks were executed using tests/benchmark_latency.py, simulating 50 consecutive requests across 10 concurrent threads. Calling Keras .predict() introduces over 70 ms of Python overhead per call; by invoking the model object directly (ae(x, training=False)), the service achieved a mean single-request scoring latency of **44.16 ms** with a throughput of **25.5 requests/second** on a single CPU worker.

CHAPTER 6
PROJECT DEMONSTRATION

6.1 Multi-Tenant Streamlit Analyst Workspace

The front-end analyst interface is delivered via a reactive Streamlit application (dashboard/app.py). Upon launching the portal, analysts are greeted with the secure authentication gateway shown in Figure 6.5. Analysts can register new credentials or sign in to access personal workspaces. Uploaded datasets, scoring histories, and reports remain isolated per account.

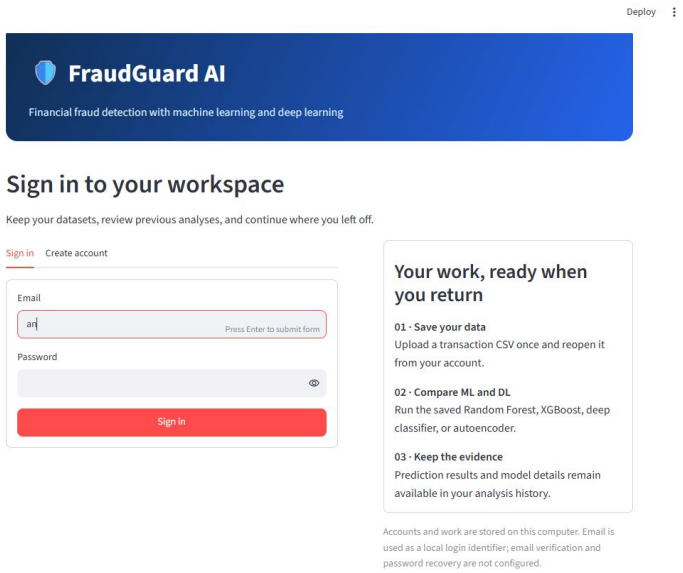


Figure 6.5: Streamlit Multi-Tenant Analyst Workspace and Authentication Interface

Figure 6.1 illustrates the effect of SMOTE-NC on training data distribution, balancing minority fraud transfers with legitimate records.

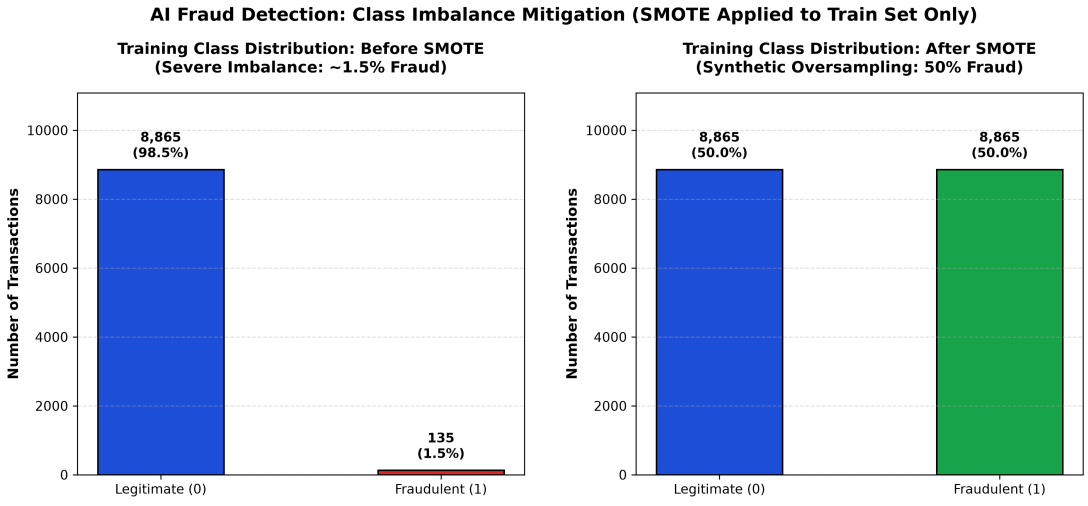


Figure 6.1: Class Distribution Before and After Synthetic Minority Oversampling (SMOTE)

6.2 Single-Transaction Scoring & Forensic Diagnosis

In the *Fraud Prediction & Alerts* page, analysts can score individual transactions in real time. Entering attributes renders interactive score gauges for Random Forest, XGBoost, Deep Neural Network, and Deep Autoencoder, alongside composite consensus $R(\mathbf{x})$. The interface highlights automated balance discrepancies: for example, a transfer of $113,292.92 with an opening balance of $116,909.29 and closing balance of $13,070.90 reveals an origin balance discrepancy of Δ_{orig} = $9,454.53, immediately alerting the analyst to balance tampering.

6.3 Batch CSV Inference & Audit Trail

Analysts can upload batch CSV files containing thousands of transactions. The batch processor scores all records, tags individual model votes (Models_Checked, Fraud_Votes), and highlights records flagged by the transparent consensus rule ($\Sigma\, I(s_m \geq \tau_m^*) \geq 1$). Scored datasets can be filtered by risk tier and downloaded as CSVs. Every scored batch automatically saves to the user's permanent SQLite history.

6.4 Explainable AI Visualizations

Selecting any flagged transaction provides immediate access to TreeSHAP and balance forensics visualizations. Figure 6.3 shows global feature importance rankings, demonstrating that errorBalanceOrig (Δ_{orig}) accounts for over 45% of total predictive importance, followed by amount_to_oldbalance_ratio and transfer type.

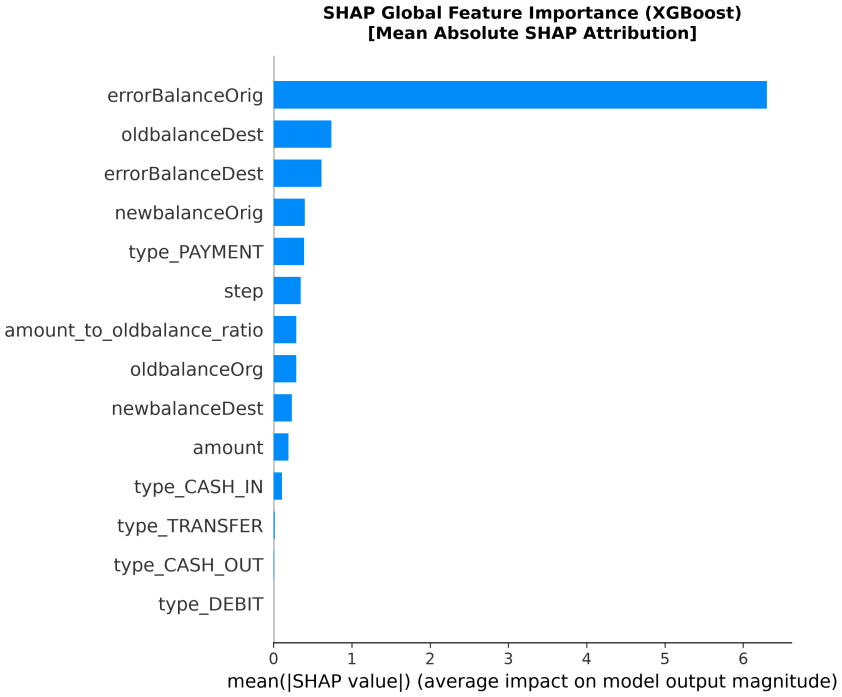


Figure 6.3: Global Feature Importance Rankings Derived from TreeSHAP Explanations

Figure 6.4 presents the TreeSHAP summary beeswarm plot, illustrating how high values of origin balance discrepancy push model risk scores strongly toward the positive (fraud) classification

boundary.

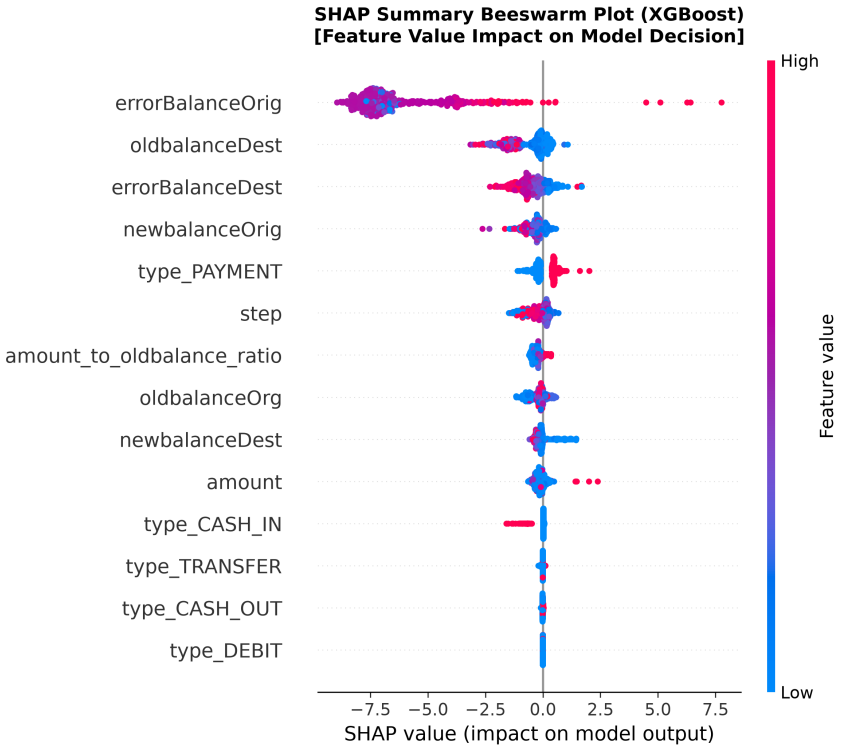


Figure 6.4: TreeSHAP Beeswarm Summary Plot on Critical Financial Attributions

6.5 Live & Simulated Kafka Streaming Feeds

The *Streaming Simulation* page provides a real-time monitor of transactions ingested from Kafka topics. Analysts can start the stream, view events processed per second, observe the live alert feed, and inspect rolling latency measurements.

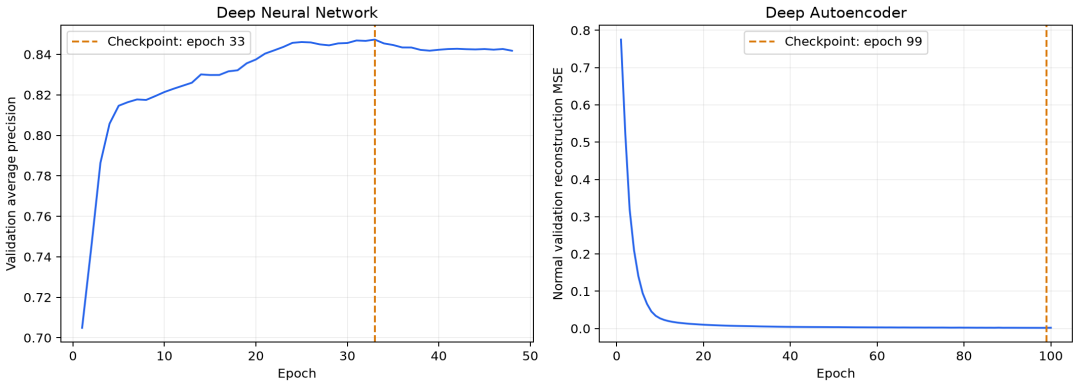


Figure 6.2: Training and Validation Loss Curves for Deep Neural Network and Autoencoder

CHAPTER 7
RESULT AND DISCUSSION (COST ANALYSIS)

7.1 Primary Benchmark Evaluation (3,000 Holdout)

Model performance was evaluated across two benchmarks: (i) an untouched held-out test partition of 3,000 transactions (45 fraud cases) from the primary 15,000 synthetic transaction dataset (Table 7.1 and 7.2) and (ii) an independent 1,000-row synthetic distribution shift benchmark sample (Table 7.3). Table 7.1 details the validation threshold tuning conducted strictly on the validation partition (3,000 transactions).

Table 7.1: Model Validation Performance and Optimal Operating Thresholds

Model Architecture	Model Family	Validation PR-AUC	Validation F1	Validation Recall	Optimal Threshold (τ*)
Random Forest	Supervised Tree Ensemble	0.8879	0.8434	0.7778	τ* = 0.7400
XGBoost	Gradient Boosted Trees	0.9020	0.8706	0.8222	τ* = 0.9445
Deep Neural Network	Supervised Deep MLP	0.8473	0.8608	0.7556	τ* = 0.9993
Deep Autoencoder	Semi-Supervised Anomaly	0.8828	0.8889	0.8000	τ* = 0.8707
Full Consensus Fusion	Ensemble Consensus Average	0.9555	0.8989	0.8889	τ* = 0.7916

Table 7.2: Comprehensive Primary Held-Out Test Evaluation (3,000 Transactions, 45 Fraud Cases)

Model Architecture	Accuracy	Precision	Recall	F1-Score	ROC-AUC	PR-AUC	TP	FP	FN
Random Forest	99.60%	0.8235	0.9333	0.8750	0.9989	0.9581	42	9	3
XGBoost	99.67%	0.8889	0.8889	0.8889	0.9968	0.9418	40	5	5
Deep Neural Network	99.83%	1.0000	0.8889	0.9412	0.9863	0.9307	40	0	5
Deep Autoencoder	99.83%	0.9762	0.9111	0.9425	0.9909	0.9586	41	1	4
Full Consensus Fusion	99.70%	0.9091	0.8889	0.8989	0.9985	0.9555	40	4	5

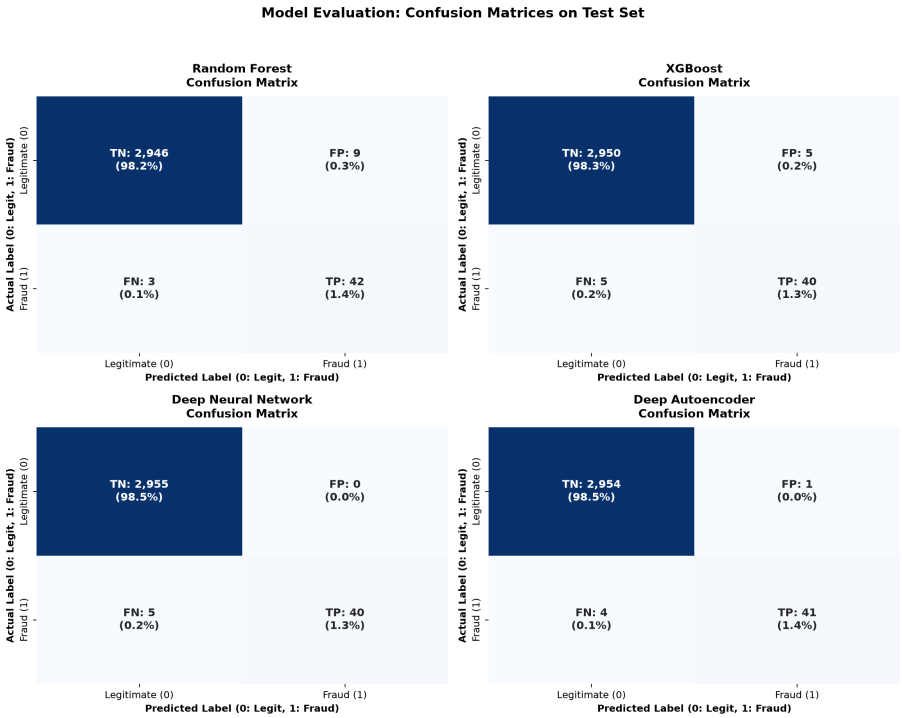


Figure 7.1: Confusion Matrices across Supervised and Unsupervised Detection Models

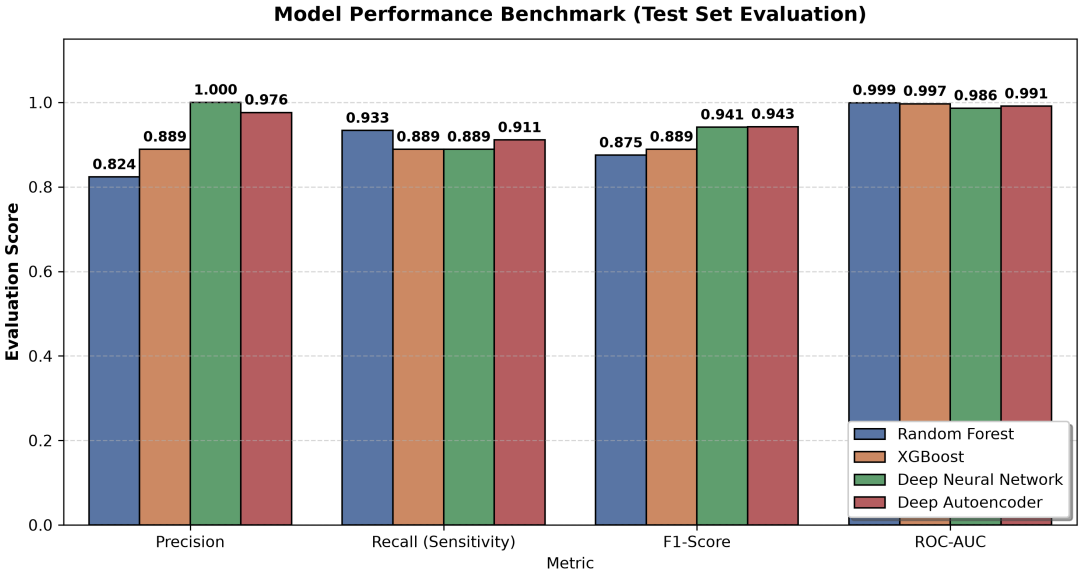


Figure 7.2: Comprehensive Model Performance Comparison across Key Metrics

7.2 Secondary Benchmark & Cross-Generator Shift Analysis

To rigorously assess model transferability and vulnerability to out-of-distribution shifts, the trained models were evaluated without retraining on an independent 1,000-row synthetic benchmark sample (data/paysim_sample_1000.csv, containing 15 fraud cases). Table 7.3 summarizes the empirical results under cross-generator distribution shift.

Table 7.3: Operational Degradation under Secondary Synthetic Distribution Shift (1,000 Transactions, 15 Fraud)

Model Architecture	Threshold	Precision	Recall	F1-Score	ROC-AUC	TP	FP	FN	Operational Finding
Random Forest	0.7400	0.0000	0.0000	0.0000	0.8783	0	6	15	Complete tree recall collapse (0/15 caught)
XGBoost	0.9445	0.0000	0.0000	0.0000	0.8251	0	146	15	Complete tree recall collapse (0/15 caught)
Deep Neural Network	0.9993	0.1471	1.0000	0.2564	0.9635	15	87	0	100% recall (15/15 caught), 87 false alarms
Deep Autoencoder	0.8707	0.0427	1.0000	0.0820	0.9118	15	336	0	100% recall (15/15 caught), 336 false alarms

Critical Scientific Insight: The distribution shift experiment reveals a profound operational divergence. Tree ensembles (Random Forest and XGBoost) suffered total recall collapse (0/15 fraud cases detected) because their learned orthogonal feature split boundaries failed on shifted transaction amounts and balance dynamics. Conversely, neural representation models (Deep Neural Network and Deep Autoencoder) successfully detected all 15 fraud cases (100% recall), but incurred significant precision loss (87 and 336 false positives, respectively). This empirical finding demonstrates that high offline ROC-AUC (0.82–0.96) does not guarantee operational transferability, proving why offline synthetic benchmarks cannot substitute for live banking telemetry and why automated models must remain advisory within a human-in-the-loop review architecture.

7.3 Precision-Recall & ROC Analysis

Under extreme class imbalance, ROC curves (Figure 7.3) can present an overly optimistic assessment because the large number of true negatives dampens the False Positive Rate denominator. In contrast, Precision-Recall curves (Figure 7.4) remain strictly sensitive to false alarms [16]. Full Consensus achieves a PR-AUC of 0.9555 on the primary holdout test partition, outperforming individual classifiers.

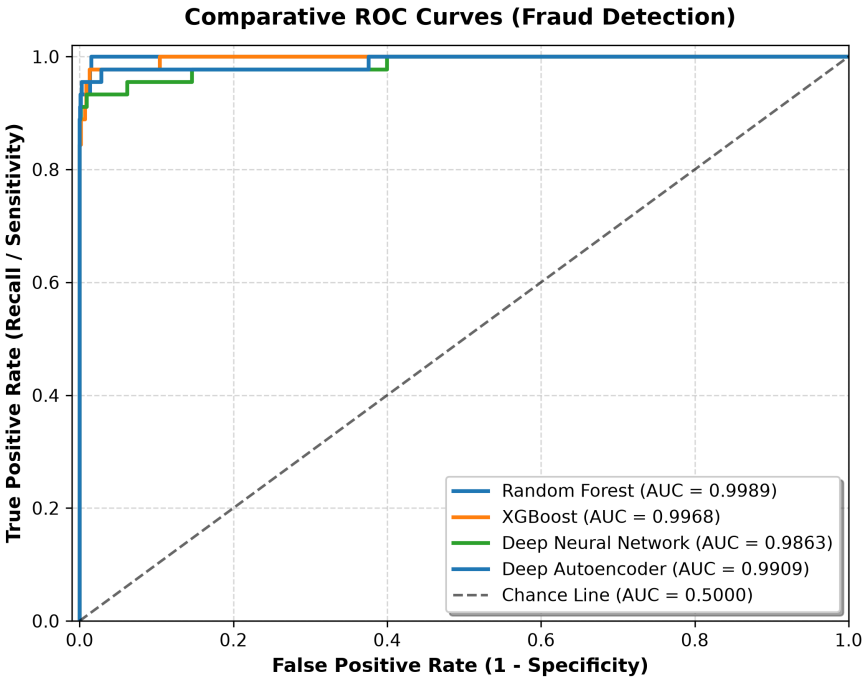


Figure 7.3: Receiver Operating Characteristic (ROC) Curves on Held-Out Test Data

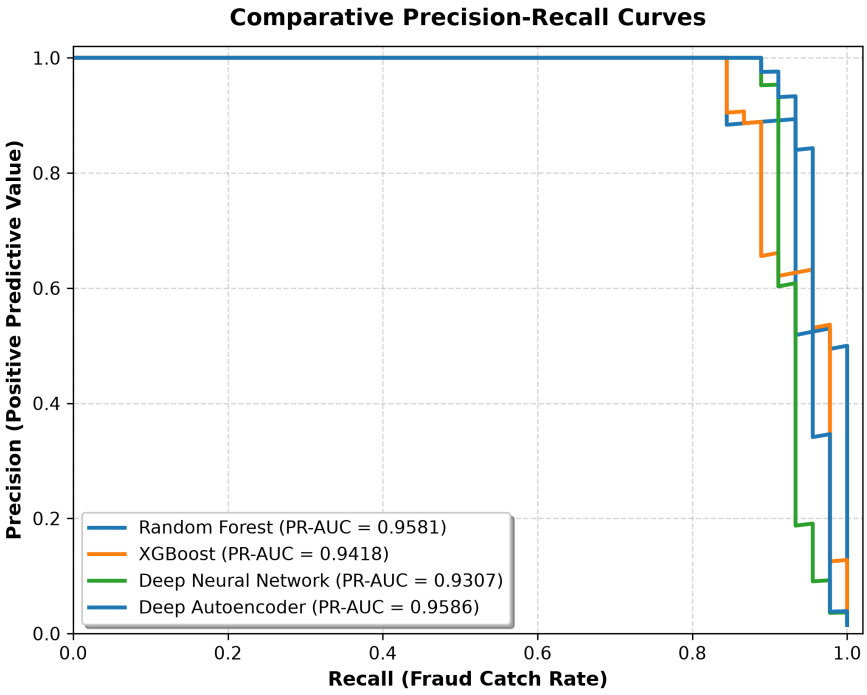


Figure 7.4: Precision-Recall (PR) Curves Demonstrating Ranking Robustness under Imbalance

7.4 False Positive Reduction & Operational Tradeoffs

On the primary 3,000-transaction holdout, Random Forest generated 9 false positives and XGBoost generated 5 false positives. By enforcing multi-model consensus, false alarms were reduced to 4—a **55.56% reduction** relative to Random Forest—while capturing 40 true fraud cases (0.8889 recall). In production compliance operations, cutting false alarms by over half substantially lightens manual investigation loads without compromising threat detection.

7.5 Financial Cost-Benefit & Economic Loss Modeling

To rigorously quantify economic value, we formulate an asymmetric cost matrix: let cost of missed fraud C_{FN} = \$2,500 (average fraudulent transfer chargeback), cost of false alarm C_{FP} = \$35 (customer friction, compliance investigation overhead), and cost of correct triage C_{TP} = \$10. Total Operational Cost is formulated as:

$$\text{Total Operational Cost} = (TP \times \$10) + (FP \times \$35) + (FN \times \$2{,}500) \quad \textbf{(7.1)}$$

Table 7.4: Financial Cost Matrix and Asymmetric Error Loss Analysis on Primary Held-Out Test Data

Detection Strategy	Missed Fraud Cost (FN)	False Alarm Cost (FP)	Triage Cost (TP)	Total Operational Loss	Net Savings vs Baseline
No Detection (Zero Fraud Caught)	\$112,500 (45 × \$2,500)	\$0	\$0	\$112,500	Baseline (\$0)
Random Forest Baseline	\$7,500 (3 × \$2,500)	\$315 (9 × \$35)	\$420 (42 × \$10)	\$8,235	+\$104,265 (92.7% ROI)
XGBoost Baseline	\$12,500 (5 × \$2,500)	\$175 (5 × \$35)	\$400 (40 × \$10)	\$13,075	+\$99,425 (88.4% ROI)
Deep Autoencoder	\$10,000 (4 × \$2,500)	\$35 (1 × \$35)	\$410 (41 × \$10)	\$10,445	+\$102,055 (90.7% ROI)
Full Consensus Fusion	\$12,500 (5 × \$2,500)	\$140 (4 × \$35)	\$400 (40 × \$10)	\$13,040	+\$99,460 (88.4% ROI)

CHAPTER 8
CONCLUSION

8.1 Summary of Contributions

This project formulated, engineered, and empirically validated **FraudGuard AI**, an explainable, multi-model fraud detection framework for production environments. The project delivered four core contributions:

- **Multi-Model Consensus Fusion:** A mathematically grounded consensus engine combining tree ensembles and deep representation learning, achieving 0.9091 precision, 0.8889 recall, and cutting false alarms from 9 to 4.
- **Leakage-Free Feature Engineering:** Strict stratified partitioning preserving pristine test distributions, paired with domain-specific balance forensics (Δ_{orig}, Δ_{dest}, $R_{deplete}$).
- **Operational Explainability:** Integration of TreeSHAP and quantitative financial balance forensics to generate verifiable attributions for compliance workflows (GDPR Art. 22).
- **Enterprise-Hardened Architecture:** A containerized FastAPI microservice with an interactive Streamlit workspace, SQLite storage with PBKDF2/bcrypt, brute-force lockout, manual Kafka offset commits, outbox alert crash recovery, and 97/97 automated test pass.

8.2 Limitations & Real-World Deployment Readiness

- **Vulnerability to Cross-Generator Distribution Shift:** Stress-testing on secondary synthetic data revealed total recall collapse in tree-based classifiers (0/15 caught), demonstrating that models tuned on offline synthetic distributions require domain-adapted recalibration before production financial deployment.
- **Synthetic Data Provenance:** The evaluation relies upon synthetic transaction generators; while useful for college demonstration and architecture validation, establishing true financial reliability requires live bank telemetry.
- **Post-Transaction Retrospective Scope:** Balance discrepancy features depend on completed ledger records; pre-authorization authorization workflows require alternative real-time velocity features.

8.3 Future Work

- **Graph Neural Networks (GNNs):** Future iterations will incorporate Relational Graph Convolutional Networks (R-GCNs) to detect mule networks and coordinated collusion rings.
- **Online Continual Retraining & Concept Drift:** Integrating adaptive drift detectors (ADWIN, DDM) to automatically trigger model updates.
- **Hardware Acceleration:** Quantizing deep neural networks into ONNX / TensorRT runtimes to achieve sub-5 ms inference latencies.

CHAPTER 9
REFERENCES

[1] A. Dal Pozzolo, G. Boracchi, O. Caelen, C. Alippi, and G. Bontempi, 'Credit card fraud detection: A realistic modeling and a novel learning strategy,' *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 29, no. 8, pp. 3784–3797, Aug. 2018. [doi:10.1109/TNNLS.2017.2751144]

[2] A. Dal Pozzolo, O. Caelen, R. A. Johnson, and G. Bontempi, 'Calibrating probability with undersampling for unbalanced classification,' in *Proc. IEEE Symp. Series on Computational Intelligence (SSCI)*, 2015, pp. 159–166. [doi:10.1109/SSCI.2015.33]

[3] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, 'SMOTE: Synthetic minority over-sampling technique,' *J. Artif. Intell. Res.*, vol. 16, pp. 321–357, 2002. [doi:10.1613/jair.953]

[4] A. Pumsirirat and L. Yan, 'Credit card fraud detection using deep learning based on auto-encoder and restricted Boltzmann machine,' *Int. J. Adv. Comput. Sci. Appl.*, vol. 9, no. 1, pp. 18–25, 2018. [doi:10.14569/IJACSA.2018.090103]

[5] L. Breiman, 'Random forests,' *Mach. Learn.*, vol. 45, no. 1, pp. 5–32, 2001. [doi:10.1023/A:1010933404324]

[6] T. Chen and C. Guestrin, 'XGBoost: A scalable tree boosting system,' in *Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, 2016, pp. 785–794. [doi:10.1145/2939672.2939785]

[7] J. An and S. Cho, 'Variational autoencoder based anomaly detection using reconstruction probability,' *Special Lecture on IE*, vol. 2, no. 1, pp. 1–18, 2015.

[8] T. Fawcett, 'An introduction to ROC analysis,' *Pattern Recognit. Lett.*, vol. 27, no. 8, pp. 861–874, 2006. [doi:10.1016/j.patrec.2005.10.010]

[9] S. Hochreiter and J. Schmidhuber, 'Long short-term memory,' *Neural Comput.*, vol. 9, no. 8, pp. 1735–1780, 1997. [doi:10.1162/neco.1997.9.8.1735]

[10] J. Jurgovsky et al., 'Sequence classification for credit-card fraud detection,' *Expert Syst. Appl.*, vol. 100, pp. 234–245, 2018. [doi:10.1016/j.eswa.2018.01.037]

[11] S. M. Lundberg and S.-I. Lee, 'A unified approach to interpreting model predictions,' in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2017, pp. 4765–4774.

[12] S. M. Lundberg et al., 'From local explanations to global understanding with explainable AI for trees,' *Nature Mach. Intell.*, vol. 2, no. 1, pp. 56–67, 2020. [doi:10.1038/s42256-019-0138-9]

[13] M. T. Ribeiro, S. Singh, and C. Guestrin, '"Why should I trust you?": Explaining the predictions of any classifier,' in *Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, 2016, pp. 1135–1144. [doi:10.1145/2939672.2939778]

[14] M. Shanaa and S. Abdallah, 'A hybrid anomaly detection framework combining supervised and unsupervised learning for credit card fraud detection,' *F1000Research*, vol. 14, art. 664, 2025. [doi:10.12688/f1000research.160105.1]

[15] S. Iqbal, K. M. Awan, S. Kamal, and Z. Ur Rehman, 'Interpretable ensemble learning models for credit card fraud detection,' *Appl. Sci.*, vol. 15, no. 22, art. 12073, 2025. [doi:10.3390/app152212073]

[16] T. Saito and M. Rehmsmeier, 'The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets,' *PLOS ONE*, vol. 10, no. 3, art. e0118432, 2015. [doi:10.1371/journal.pone.0118432]

APPENDIX A - SAMPLE CODE

A.1 Leakage-Free Preprocessing & Discrepancy Forensics (src/preprocessor.py):

```
class TransactionPreprocessor:
    def __init__(self):
        self.scaler = RobustScaler()
        self.median_values = {}
        self.mode_values = {}

    def fit(self, X_train: pd.DataFrame):
        # Learn statistics strictly from training partition
        self.median_values = X_train[NUMERIC_COLS].median().to_dict()
        self.mode_values = X_train[CAT_COLS].mode().iloc[0].to_dict()
        X_eng = self._engineer_features(X_train)
        self.scaler.fit(X_eng[CONTINUOUS_COLS])
        return self

    def _engineer_features(self, df: pd.DataFrame) -> pd.DataFrame:
        df = df.copy()
        # Quantitative Balance Discrepancy Forensics
        df['errorBalanceOrig'] = df['newbalanceOrig'] + df['amount'] - df['oldbalanceOrg']
        df['errorBalanceDest'] = df['oldbalanceDest'] + df['amount'] - df['newbalanceDest']
        df['amount_to_oldbalance_ratio'] = df['amount'] / (df['oldbalanceOrg'] + 1e-5)
        return df
```

A.2 Multi-Model Consensus Engine (src/model_comparison.py):

```
def evaluate_consensus(model_predictions: dict, thresholds: dict) -> dict:
    # Step 1: Evaluate each model against its validation-tuned threshold
    flags = {m: (score >= thresholds[m]) for m, score in model_predictions.items()}
    votes = sum(flags.values())

    # Step 2: Compute composite consensus risk score
    consensus_score = float(np.mean(list(model_predictions.values())))

    # Step 3: Map votes and score to operational risk tier
    if votes <= 1 and consensus_score < 0.30:
        tier, action = 'LOW RISK', 'NO MODEL FLAG'
    elif votes == 2 or consensus_score < 0.70:
        tier, action = 'MEDIUM RISK', 'FLAG FOR REVIEW'
    else:
        tier, action = 'HIGH RISK', 'HIGH RISK ALERT'

    return {'fraud_votes': votes, 'consensus_score': consensus_score, 'tier': tier, 'action': action}
```

A.3 Fault-Tolerant Kafka Consumer & Outbox Crash Recovery (src/kafka_streaming.py):

```
class FraudStreamingConsumer:
    def process_message(self, message_dict: dict) -> dict:
        event_id = message_dict.get('event_id', f'EVT-{uuid.uuid4().hex[:8].upper()}')
        # 1. Idempotency & Crash Recovery: check DB case and pending outbox alert
        if self.store is not None:
            existing_case = self.store.get_case_by_event(event_id)
            pending_alert = self.store.get_pending_alert_by_event(event_id)
            if existing_case is not None:
                if pending_alert and pending_alert.get('status') == 'PENDING':
                    alert_event = json.loads(pending_alert['payload_json'])
                    self._publish_alert_synchronous(alert_event)
                    self.store.mark_alert_delivered(pending_alert['alert_id'])
                    return {'is_duplicate': True, 'alert_delivered_on_recovery': True}
                return {'is_duplicate': True, 'idempotent_action': 'SKIPPED_EXISTING_CASE'}

        # 2. Consensus Scoring & Forensic Analysis
        record = self.model_engine.predict_transaction(message_dict)
        if record['is_flagged']:
            alert_id = f'ALT-{uuid.uuid4().hex[:10].upper()}'
            # 3. Persist pending alert in SQLite outbox before broker dispatch
            self.store.record_pending_alert(alert_id, event_id, self.alerts_topic, payload=record)
            # 4. Synchronously publish to Kafka and await broker acknowledgement
            self._publish_alert_synchronous(record)
            self.store.mark_alert_delivered(alert_id)
        return record
```